

1. A hospital database has PATIENT(PID, Name, DOB, DoctorID) and DOCTOR(DID, Name, Specialization). Write relational algebra to find all patients treated by a cardiologist.

Aim: To retrieve the details of all patients who are treated by a cardiologist by joining the PATIENT and DOCTOR tables.

Query

```
SELECT P.PID, P.Name
```

```
FROM PATIENT P
```

```
JOIN DOCTOR D
```

```
ON P.DoctorID = D.DID
```

```
WHERE D.Specialization = 'Cardiologist';
```

```
MySQL 8.0 Command Line Cli x + v
-> Name VARCHAR(50),
-> Specialization VARCHAR(50)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE PATIENT
-> (
-> PID INT PRIMARY KEY,
-> Name VARCHAR(50),
-> DOB DATE,
-> DoctorID INT,
-> FOREIGN KEY (DoctorID) REFERENCES DOCTOR(DID)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO DOCTOR VALUES
-> (101, 'Ramesh', 'Cardiologist'),
-> (102, 'Priya', 'Neurologist'),
-> (103, 'Arun', 'Cardiologist');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO DOCTOR VALUES
-> (101, 'Ramesh', 'Cardiologist'),
-> (102, 'Priya', 'Neurologist'),
-> (103, 'Arun', 'Cardiologist');
ERROR 1062 (23000): Duplicate entry '101' for key 'doctor.PRIMARY'
mysql> SELECT * FROM DOCTOR;
+-----+-----+-----+
| DID | Name | Specialization |
+-----+-----+-----+
| 101 | Ramesh | Cardiologist |
| 102 | Priya | Neurologist |
| 103 | Arun | Cardiologist |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM PATIENT;
Empty set (0.01 sec)

mysql> SELECT P.PID, P.Name
-> FROM PATIENT P
-> JOIN DOCTOR D
-> ON P.DoctorID=D.DID
-> WHERE D.Specialization='Cardiologist';
Empty set (0.01 sec)
```

1. An e-commerce platform needs to find all customers who placed orders in January 2024 but NOT in February 2024. Write the SQL query using set operations.

Aim: To identify customers who placed orders in January 2024 but did not place any orders in February 2024.

Query (MySQL)

SELECT DISTINCT CustomerID

FROM ORDERS

```
WHERE OrderDate BETWEEN '2024-01-01' AND '2024-01-31'

AND CustomerID NOT IN

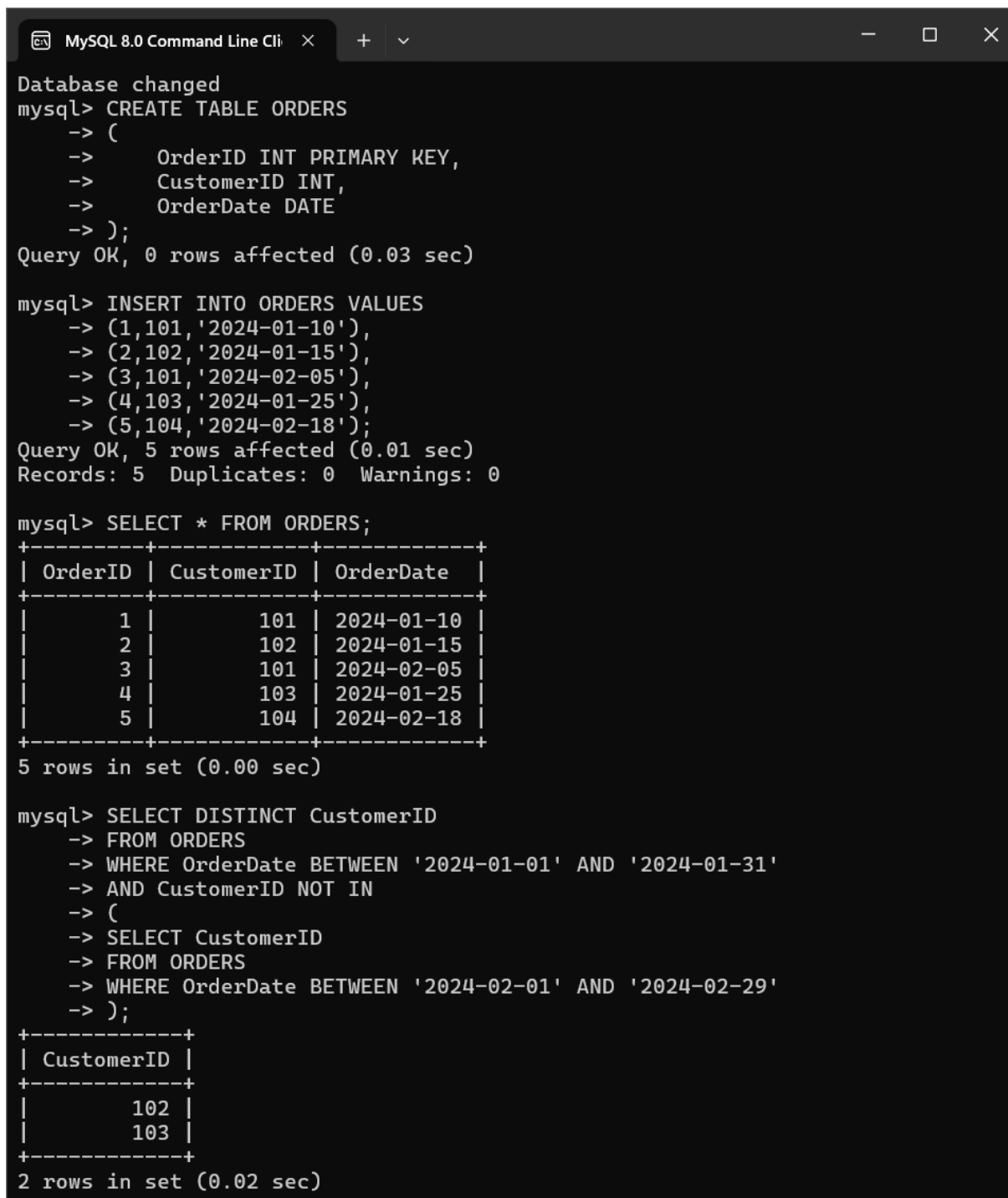
(

    SELECT CustomerID

    FROM ORDERS

    WHERE OrderDate BETWEEN '2024-02-01' AND '2024-02-29'

);
```



```
MySQL 8.0 Command Line Cli x + v
Database changed
mysql> CREATE TABLE ORDERS
-> (
->     OrderID INT PRIMARY KEY,
->     CustomerID INT,
->     OrderDate DATE
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO ORDERS VALUES
-> (1,101,'2024-01-10'),
-> (2,102,'2024-01-15'),
-> (3,101,'2024-02-05'),
-> (4,103,'2024-01-25'),
-> (5,104,'2024-02-18');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ORDERS;
+-----+-----+-----+
| OrderID | CustomerID | OrderDate |
+-----+-----+-----+
| 1 | 101 | 2024-01-10 |
| 2 | 102 | 2024-01-15 |
| 3 | 101 | 2024-02-05 |
| 4 | 103 | 2024-01-25 |
| 5 | 104 | 2024-02-18 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT DISTINCT CustomerID
-> FROM ORDERS
-> WHERE OrderDate BETWEEN '2024-01-01' AND '2024-01-31'
-> AND CustomerID NOT IN
-> (
->     SELECT CustomerID
->     FROM ORDERS
->     WHERE OrderDate BETWEEN '2024-02-01' AND '2024-02-29'
-> );
+-----+
| CustomerID |
+-----+
| 102 |
| 103 |
+-----+
2 rows in set (0.02 sec)
```

3. A university database contains COURSE(CID, Name, Credits) and ENROLLMENT(SID, CID, Grade). Create a view showing each student's GPA calculated from their grades.

Aim: To create a SQL view that calculates and displays the GPA of each student based on grades and course credits.

Query

```
CREATE VIEW Student_GPA AS
SELECT
    E.SID,
    ROUND(SUM(E.Grade * C.Credits) / SUM(C.Credits), 2) AS GPA
FROM ENROLLMENT E
JOIN COURSE C
ON E.CID = C.CID
GROUP BY E.SID;
```

Display the View

```
SELECT * FROM Student_GPA;
```

```
MySQL 8.0 Command Line Cli x + v
-> (2,101,7),
-> (2,102,9);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| CID | Name | Credits |
+-----+-----+-----+
| 101 | DBMS | 4 |
| 102 | Java | 3 |
| 103 | Python | 3 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM ENROLLMENT;
+-----+-----+-----+
| SID | CID | Grade |
+-----+-----+-----+
| 1 | 101 | 9.00 |
| 1 | 102 | 8.00 |
| 1 | 103 | 10.00 |
| 2 | 101 | 7.00 |
| 2 | 102 | 9.00 |
+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> CREATE VIEW Student_GPA AS
-> SELECT
-> E.SID,
-> ROUND(SUM(E.Grade*C.Credits)/SUM(C.Credits),2) AS GPA
-> FROM ENROLLMENT E
-> JOIN COURSE C
-> ON E.CID=C.CID
-> GROUP BY E.SID;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Student_GPA;
+-----+-----+
| SID | GPA |
+-----+-----+
| 1 | 9.00 |
| 2 | 7.86 |
+-----+-----+
2 rows in set (0.01 sec)
```

4. For a banking system with ACCOUNT(AccNo, CustID, Type, Balance), write relational algebra to find all customers who have both a savings and a current account .

Aim :To identify customers who have both Savings and Current accounts.

Query

SELECT CustID

FROM ACCOUNT

GROUP BY CustID

HAVING COUNT(DISTINCT Type) = 2;

```
MySQL 8.0 Command Line Cli x + v

mysql>
mysql> USE BankDB;
Database changed
mysql>
mysql> CREATE TABLE ACCOUNT
-> (
->     AccNo INT PRIMARY KEY,
->     CustID INT,
->     Type VARCHAR(20),
->     Balance DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO ACCOUNT VALUES
-> (1001,101,'Savings',20000),
-> (1002,101,'Current',15000),
-> (1003,102,'Savings',18000),
-> (1004,103,'Current',25000),
-> (1005,103,'Savings',22000);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM ACCOUNT;
+-----+-----+-----+-----+
| AccNo | CustID | Type   | Balance |
+-----+-----+-----+-----+
| 1001  | 101    | Savings | 20000.00 |
| 1002  | 101    | Current | 15000.00 |
| 1003  | 102    | Savings | 18000.00 |
| 1004  | 103    | Current | 25000.00 |
| 1005  | 103    | Savings | 22000.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT CustID
-> FROM ACCOUNT
-> GROUP BY CustID
-> HAVING COUNT(DISTINCT Type)=2;
+-----+
| CustID |
+-----+
| 101    |
| 103    |
+-----+
2 rows in set (0.01 sec)
```

5. A retail store wants to identify products that have never been ordered. Using PRODUCT(PID, Name) and ORDER_ITEM(OID, PID, Qty), write the SQL query using NOT IN and NOT EXISTS."

Aim: To identify products that have never been ordered using NOT IN and NOT EXISTS.

Query using NOT IN

```
SELECT PID, Name
```

```
FROM PRODUCT
```

```
WHERE PID NOT IN
```

```
(
```

```
    SELECT PID
```

```
    FROM ORDER_ITEM
```

```
);
```

Query using NOT EXISTS

```
SELECT P.PID, P.Name
```

```
FROM PRODUCT P
```

```
WHERE NOT EXISTS
```

```
(
```

```
    SELECT *
```

```
    FROM ORDER_ITEM O
```

```
    WHERE O.PID = P.PID
```

```
);
```

```
mysql> INSERT INTO ORDER_ITEM VALUES
-> (101,1,2),
-> (102,2,1),
-> (103,1,1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM PRODUCT;
```

PID	Name
1	Laptop
2	Mouse
3	Keyboard
4	Monitor

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM ORDER_ITEM;
```

OID	PID	Qty
101	1	2
102	2	1
103	1	1

3 rows in set (0.00 sec)

```
mysql> SELECT PID,Name
-> FROM PRODUCT
-> WHERE PID NOT IN
-> (
-> SELECT PID
-> FROM ORDER_ITEM
-> );
```

PID	Name
3	Keyboard
4	Monitor

2 rows in set (0.01 sec)

```
mysql> SELECT P.PID,P.Name
-> FROM PRODUCT P
-> WHERE NOT EXISTS
-> (
-> SELECT *
-> FROM ORDER_ITEM O
-> WHERE O.PID=P.PID
-> );
```

PID	Name
3	Keyboard
4	Monitor